```
!pip install tenseal
```

```
Collecting tenseal
  Downloading tenseal-0.3.18-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (10 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from tenseal) (2.1.3)
Downloading tenseal-0.3.18-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (5.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.2/5.2 MB 17.3 MB/s eta 0:00:00
Installing collected packages: tenseal
Successfully installed tenseal-0.3.18
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# -*- coding: utf-8 -*-
"""
Script Otimizado para Teste da PoC (Fase 3): Criptografia Homomórfica com TenSEAL/CKKS
Modelo: Autoencoder Tabular (PyTorch) + Aprendizado Federado (FedAvg) Paralelizado
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import concurrent.futures
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

import tenseal as ts

# ========================================
# NOVO: FUNÇÃO PARA FIXAR O DETERMINISMO
# ========================================
def set_seed(seed=42):
    import random
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        # Forçar operações determinísticas na GPU (pode reduzir performance)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

# Aplicar a semente antes de qualquer coisa
set_seed(42)

# Configurações de estilo e dispositivo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 5)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")


print("\n--- 1. Carregamento e Preparação dos Dados ---\n")
try:
    # Atualizado o caminho do CSV para o Google Drive
    file_path = '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv'
    df = pd.read_csv(file_path)

    # Sub-sample para FHE (Foco em manter os dados de fraude e uma amostra de normais)
    if len(df) > 100000:
        print("Realizando amostragem (100k linhas) para viabilidade do FHE...")
        df_fraudes = df[df['isFraud'] == 1]
        df_normais = df[df['isFraud'] == 0].sample(n=(100000 - len(df_fraudes)), random_state=42)
        df = pd.concat([df_normais, df_fraudes]).sample(frac=1, random_state=42).reset_index(drop=True)

except FileNotFoundError:
    raise FileNotFoundError(f"ERRO: O arquivo '{file_path}' não foi encontrado. Verifique se o Google Drive está montado e
```

```
df = df.drop(columns=['nameOrig', 'nameDest', 'isFlaggedFraud']).dropna(subset=['isFraud'])

print("Aplicando Engenharia de Features...")
df['amount_log'] = np.log1p(df['amount'])
df['errorBalanceOrig'] = df['newbalanceOrig'] + df['amount'] - df['oldbalanceOrg']

# One-Hot Encoding e separação
df_ml = pd.get_dummies(df, columns=['type'], drop_first=True)
X = df_ml.drop(columns=['isFraud']).astype(np.float32)
y = df_ml['isFraud'].astype(np.int32)

# Correção de Data Leakage: Split antes de escalar
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X.values, y.values, test_size=0.2, random_state=42, stratify=y.values
)

# Fit apenas no X_train, transform no X_train e X_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

input_dim = X_train.shape[1]
print(f"Dimensão de entrada (Features): {input_dim}")


class TabularAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(TabularAutoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 8),
            nn.ReLU()
        )
        # OMITIDO a Sigmoid() no decoder pois o StandardScaler gera valores fora de [0,1]
        self.decoder = nn.Sequential(
            nn.Linear(8, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))


print("\n--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---\n")
print("Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...")
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.global_scale = 2**40
context.generate_galois_keys()
print("-> Contexto CKKS gerado com sucesso!")


print("\n--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---\n")
NUM_CLIENTES = 3

# Filtra apenas dados normais para treinar o Autoencoder
X_train_normal = X_train[y_train == 0]
tamanho_chunk = len(X_train_normal) // NUM_CLIENTES
clientes_X = [X_train_normal[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelo_global = TabularAutoencoder(input_dim).to(device)
criterion = nn.MSELoss()

def treinar_cliente_e_criptografar(cliente_id, dados_X, state_dict_global, epochs=3, batch_size=64):
    """Função que roda em paralelo: Treina, achata os pesos nativamente e os encripta."""
    inicio = time.time()

    # 1. Setup local
    modelo_local = TabularAutoencoder(input_dim).to(device)
    modelo_local.load_state_dict(state_dict_global)
    modelo_local.train()
    optimizer = optim.Adam(modelo_local.parameters(), lr=0.01)

    # Otimização: Pin_memory acelera transferãncia para GPU
    dataset = TensorDataset(torch.tensor(dados_X, dtype=torch.float32), torch.tensor(dados_X, dtype=torch.float32))

    # NOVO: Garantir que o DataLoader use a mesma seed para o shuffle
    generator = torch.Generator(device='cpu') # Generator para dataloader sempre no CPU
    generator.manual_seed(42 + cliente_id) # Seed ónica mas determinística por cliente
```

```
        loader = DataLoader(
            dataset,
            batch_size=batch_size,
            shuffle=True,
            pin_memory=(device.type == 'cuda'),
            generator=generator # Aplica o generator determinístico
        )

        # 2. Treinamento
        for _ in range(epochs):
            for data, target in loader:
                data, target = data.to(device, non_blocking=True), target.to(device, non_blocking=True)
                optimizer.zero_grad()
                output = modelo_local(data)
                loss = criterion(output, target)
                loss.backward()
                optimizer.step()

        # 3. Extração Otimizada de Pesos (Vetorização C++ nativa do PyTorch)
        pesos_planos = parameters_to_vector(modelo_local.parameters()).detach().cpu().numpy().tolist()

        # 4. Criptografia Homomórfica
        pesos_cifrados = ts.ckks_vector(context, pesos_planos)

        tempo_total = time.time() - inicio
        print(f"[Cliente {cliente_id+1}] Treino e Criptografia concluídos em {tempo_total:.2f}s.")
        return pesos_cifrados


    # Usando ThreadPoolExecutor para simular os clientes operando simultaneamente
    pesos_criptografados_clientes = []
    state_dict_atual = {k: v.cpu() for k, v in modelo_global.state_dict().items()}

    with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_CLIENTES) as executor:
        futures = [
            executor.submit(treinar_cliente_e_criptografar, i, clientes_X[i], state_dict_atual)
            for i in range(NUM_CLIENTES)
        ]
        for future in concurrent.futures.as_completed(futures):
            pesos_criptografados_clientes.append(future.result())

    print("\n--- 4. Agregação Federada CEGA (Servidor Central) ---\n")
    print("Servidor calculando operações sobre dados cifrados (FHE)...")
    inicio_agregracao = time.time()

    # Soma homomórfica
    soma_cifrada = pesos_criptografados_clientes[0]
    for i in range(1, NUM_CLIENTES):
        soma_cifrada += pesos_criptografados_clientes[i]

    # Média homomórfica
    media_cifrada = soma_cifrada * (1.0 / NUM_CLIENTES)

    tempo_agreg = time.time() - inicio_agregracao
    print(f"-> Agregação matemática FHE finalizada em {tempo_agreg:.2f}s.")


    print("\n--- 5. Descriptografia e Inferência ---\n")
    # Cliente decifra e restaura a topologia
    pesos_planos_agregados = media_cifrada.decrypt()

    # Restauração otimizada dos pesos para o modelo global
    tensor_pesos = torch.tensor(pesos_planos_agregados, dtype=torch.float32).to(device)
    vector_to_parameters(tensor_pesos, modelo_global.parameters())
    print("-> Modelo Global atualizado com os pesos agregados!")

    # AVALIAÇÃO
    modelo_global.eval()
    inicio_inferencia = time.time()

    with torch.no_grad():
        X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
        reconstrucoes = modelo_global(X_test_tensor)
        erros_reconstrucao = torch.mean((X_test_tensor - reconstrucoes) ** 2, dim=1).cpu().numpy()

    fim_inferencia = time.time()
    tempo_inferencia = fim_inferencia - inicio_inferencia

    # Limiar de anomalia (Fraude) fixado no Percentil 95
    threshold = np.percentile(erros_reconstrucao, 95)
    y_pred = (erros_reconstrucao > threshold).astype(int)
```

```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, erros_reconstrucao)

print("\n>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):")
print(f"Acurácia Geral      : {acc * 100:.2f}%")
print(f"Precisão (Precision): {prec * 100:.2f}%")
print(f"Revocação (Recall)  : {rec * 100:.2f}%")
print(f"F1-Score            : {f1 * 100:.2f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"Throughput de Infe. : {len(X_test)/tempo_inferencia:.2f} TPS")

# Plotagem de Resultados
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribuição de Erros
sns.histplot(erros_reconstrucao[y_test == 0], color='#2ecc71', bins=50, alpha=0.6, label='Normal', ax=axes[0])
sns.histplot(erros_reconstrucao[y_test == 1], color='#e74c3c', bins=50, alpha=0.6, label='Fraude', ax=axes[0])
axes[0].axvline(threshold, color='black', linestyle='--', linewidth=2, label='Limiar (Threshold)')
axes[0].set_yscale('log')
axes[0].set_title('Erro de Reconstrução FHE (Escala Log)', fontsize=12)
axes[0].set_xlabel('Erro Quadrático Médio (MSE)')
axes[0].set_ylabel('Frequência (Log)')
axes[0].legend()

# Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'],
            cbar=False, annot_kws={"size": 14})
axes[1].set_title('Matriz de Confusão (Agregação Segura)', fontsize=12)
axes[1].set_xlabel('Predito')
axes[1].set_ylabel('Real')

plt.tight_layout()
plt.show()

print("\nFim do processamento (Fase 3 Otimizada)!")
```

```
Dispositivo PyTorch: cuda

--- 1. Carregamento e Preparação dos Dados ---

Realizando amostragem (100k linhas) para viabilidade do FHE...
Aplicando Engenharia de Features...
Dimensão de entrada (Features): 12

--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---

Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...
-> Contexto CKKS gerado com sucesso!

--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---

[Cliente 2] Treino e Criptografia concluídos em 10.32s.
[Cliente 3] Treino e Criptografia concluídos em 10.36s.
[Cliente 1] Treino e Criptografia concluídos em 10.57s.

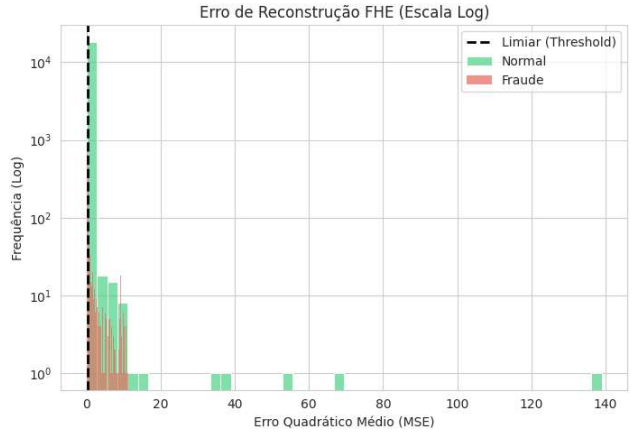
--- 4. Agregação Federada CEGA (Servidor Central) ---

Servidor calculando operações sobre dados cifrados (FHE)...
-> Agregação matemática FHE finalizada em 0.00s.

--- 5. Descriptografia e Inferência ---

-> Modelo Global atualizado com os pesos agregados!

>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):
Acurácia Geral      : 91.05%
Precisão (Precision): 42.70%
Revocação (Recall)  : 25.99%
F1-Score            : 32.31%
ROC-AUC             : 0.8258
Throughput de Infe. : 285990.22 TPS
```






```
Fim do processamento (Fase 3 Otimizada)!
```

```
!pip install tenseal
```

```
Requirement already satisfied: tenseal in /usr/local/lib/python3.13/dist-packages (0.3.18)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from tenseal) (2.1.3)
```

```
# -*- coding: utf-8 -*-
"""
Script Otimizado para Teste da PoC (Fase 3): Criptografia Homomórfica com TenSEAL/CKKS
Dataset: Credit Card Fraud (Kaggle)
Modelo: Autoencoder Tabular (PyTorch) + Aprendizado Federado (FedAvg) Paralelizado
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import concurrent.futures
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

try:
    import tenseal as ts
except ImportError:
    print("ERRO: TenSEAL não está instalado no seu ambiente.")
    print("Por favor, execute: pip install tenseal")
    exit()

# ========================================
# FUNÇÃO PARA FIXAR O DETERMINISMO
# ========================================
def set_seed(seed=42):
    import random
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        # Forçar operações determinísticas na GPU (pode reduzir performance)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

# Aplicar a semente antes de qualquer coisa
set_seed(42)

# Configurações de estilo e dispositivo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 5)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")

print("\n--- 1. Carregamento e Preparação dos Dados ---\n")
file_path = '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv'

try:
    df = pd.read_csv(file_path)
    print("Arquivo 'creditcard.csv' carregado com sucesso!")
except FileNotFoundError:
    print(f"Aviso: O arquivo '{file_path}' não foi encontrado.")
    print("Criando dados sintéticos no formato Credit Card para simulação FHE.")
    # Fallback sintético
    df = pd.DataFrame(np.random.randn(10000, 28), columns=[f'V{i}' for i in range(1, 29)])
    df['Time'] = np.random.uniform(0, 172792, 10000)
    df['Amount'] = np.abs(np.random.randn(10000)) * 100
    df['Class'] = np.random.choice([0, 1], 10000, p=[0.995, 0.005])

# Sub-sample para FHE (Foco em manter os dados de fraude e uma amostra de normais)
# O CKKS e a vetorização de pesos da rede neural podem consumir muita RAM.
# É uma boa prática reduzir a base normal para testes rápidos de conceito.
if len(df) > 100000:
    print("Realizando amostragem (100k linhas) para viabilidade do FHE...")
    df_fraudes = df[df['Class'] == 1]
    df_normais = df[df['Class'] == 0].sample(n=(100000 - len(df_fraudes)), random_state=42)
    df = pd.concat([df_normais, df_fraudes]).sample(frac=1, random_state=42).reset_index(drop=True)

# Remover Time (Ruído para autoencoder tabular)
if 'Time' in df.columns:
    df = df.drop(columns=['Time'])

df = df.dropna(subset=['Class'])

print("Aplicando Engenharia de Features...")
# Proteção para o log1p
if 'Amount' in df.columns:
    df['Amount'] = df['Amount'].apply(lambda x: max(x, 0))
    df['amount_log'] = np.log1p(df['Amount'])
    df = df.drop(columns=['Amount'])

# Separação
X = df.drop(columns=['Class']).values.astype(np.float32)
y = df['Class'].values.astype(np.int32)
```

```
# Correção de Data Leakage: Split Duplo (Treino, Validação, Teste)
X_temp, X_test_raw, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train_raw, X_val_raw, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp # 0.25 de 0.8 = 0.2
)

# Fit apenas no X_train, transform no X_train, X_val e X_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_val = scaler.transform(X_val_raw)
X_test = scaler.transform(X_test_raw)

input_dim = X_train.shape[1]
print(f"Dimensão de entrada (Features): {input_dim}")

class TabularAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(TabularAutoencoder, self).__init__()
        # Para base PCA de cartão de crédito (~29 features), uma compressão simples para FHE
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 14),
            nn.ReLU(),
            nn.Linear(14, 8),
            nn.ReLU()
        )
        # OMITIDO a Sigmoid() no decoder pois o StandardScaler gera valores fora de [0,1]
        self.decoder = nn.Sequential(
            nn.Linear(8, 14),
            nn.ReLU(),
            nn.Linear(14, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

print("\n--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---\n")
print("Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...")
# O grau do polinômio (8192) dita a segurança e a capacidade de packing (quantos pesos cabem em um cipher).
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.global_scale = 2**40
context.generate_galois_keys()
print("-> Contexto CKKS gerado com sucesso!")

print("\n--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---\n")
NUM_CLIENTES = 3

# Filtra apenas dados normais para treinar o Autoencoder
X_train_normal = X_train[y_train == 0]
tamanho_chunk = len(X_train_normal) // NUM_CLIENTES
clientes_X = [X_train_normal[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelo_global = TabularAutoencoder(input_dim).to(device)
criterion = nn.MSELoss()

def treinar_cliente_e_criptografar(cliente_id, dados_X, state_dict_global, epochs=3, batch_size=64):
    """Função que roda em paralelo: Treina, achata os pesos nativamente e os encripta."""
    inicio = time.time()

    # 1. Setup local
    modelo_local = TabularAutoencoder(input_dim).to(device)
    modelo_local.load_state_dict(state_dict_global)
    modelo_local.train()
    optimizer = optim.Adam(modelo_local.parameters(), lr=0.01)

    # Otimização: Pin_memory acelera transferência para GPU
    dataset = TensorDataset(torch.tensor(dados_X, dtype=torch.float32), torch.tensor(dados_X, dtype=torch.float32))

    # NOVO: Garantir que o DataLoader use a mesma seed para o shuffle
    generator = torch.Generator(device='cpu') # Generator para dataloader sempre no CPU
    generator.manual_seed(42 + cliente_id) # Seed única mas determinística por cliente

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=(device.type == 'cuda'),
```

```
            generator=generator # Aplica o generator determinístico
        )

        # 2. Treinamento
        for _ in range(epochs):
            for data, target in loader:
                data, target = data.to(device, non_blocking=True), target.to(device, non_blocking=True)
                optimizer.zero_grad()
                output = modelo_local(data)
                loss = criterion(output, target)
                loss.backward()
                optimizer.step()

        # 3. Extração Otimizada de Pesos (Vetorização C++ nativa do PyTorch)
        pesos_planos = parameters_to_vector(modelo_local.parameters()).detach().cpu().numpy().tolist()

        # 4. Criptografia Homomórfica
        pesos_cifrados = ts.ckks_vector(context, pesos_planos)

        tempo_total = time.time() - inicio
        print(f"[Cliente {cliente_id+1}] Treino e Criptografia concluídos em {tempo_total:.2f}s.")
        return pesos_cifrados

    # Usando ThreadPoolExecutor para simular os clientes operando simultaneamente
    pesos_criptografados_clientes = []
    state_dict_atual = {k: v.cpu() for k, v in modelo_global.state_dict().items()}

    with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_CLIENTES) as executor:
        futures = [
            executor.submit(treinar_cliente_e_criptografar, i, clientes_X[i], state_dict_atual)
            for i in range(NUM_CLIENTES)
        ]
        for future in concurrent.futures.as_completed(futures):
            pesos_criptografados_clientes.append(future.result())

    print("\n--- 4. Agregação Federada CEGA (Servidor Central) ---\n")
    print("Servidor calculando operações sobre dados cifrados (FHE)...")
    inicio_agregracao = time.time()

    # Soma homomórfica
    soma_cifrada = pesos_criptografados_clientes[0]
    for i in range(1, NUM_CLIENTES):
        soma_cifrada += pesos_criptografados_clientes[i]

    # Média homomórfica
    media_cifrada = soma_cifrada * (1.0 / NUM_CLIENTES)

    tempo_agreg = time.time() - inicio_agregracao
    print(f"-> Agregação matemática FHE finalizada em {tempo_agreg:.2f}s.")

    print("\n--- 5. Descriptografia e Inferência ---\n")
    # Cliente decifra e restaura a topologia
    pesos_planos_agregados = media_cifrada.decrypt()

    # Restauração otimizada dos pesos para o modelo global
    tensor_pesos = torch.tensor(pesos_planos_agregados, dtype=torch.float32).to(device)
    vector_to_parameters(tensor_pesos, modelo_global.parameters())
    print("-> Modelo Global atualizado com os pesos agregados!")

    print("\n--- 5.1 Otimização de Limiar (Base de Validação) ---\n")
    modelo_global.eval()

    with torch.no_grad():
        X_val_tensor = torch.tensor(X_val, dtype=torch.float32).to(device)
        reconstrucoes_val = modelo_global(X_val_tensor)
        erros_reconstrucao_val = torch.mean((X_val_tensor - reconstrucoes_val) ** 2, dim=1).cpu().numpy()

    # Limiar de anomalia (Fraude) fixado no Percentil 98 USANDO APENAS A VALIDAÇÃO
    threshold = np.percentile(erros_reconstrucao_val, 98)
    print(f"Limiar de Anomalia Definido (P98 na Validação): {threshold:.4f}")

    print("\n--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---\n")
    inicio_inferencia = time.time()

    with torch.no_grad():
        X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
        reconstrucoes = modelo_global(X_test_tensor)
        erros_reconstrucao = torch.mean((X_test_tensor - reconstrucoes) ** 2, dim=1).cpu().numpy()

    fim_inferencia = time.time()
    tempo_inferencia = fim_inferencia - inicio_inferencia
```

```
# Aplicando o limiar descoberto na validação sobre os dados de teste
y_pred = (erros_reconstrucao > threshold).astype(int)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, erros_reconstrucao)

print("\n>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):")
print(f"Acurácia Geral      : {acc * 100:.2f}%")
print(f"Precisão (Precision): {prec * 100:.2f}%")
print(f"Revocação (Recall)  : {rec * 100:.2f}%")
print(f"F1-Score            : {f1 * 100:.2f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"Throughput de Infe. : {len(X_test)/tempo_inferencia:.2f} TPS")

# Plotagem de Resultados
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribuição de Erros
sns.histplot(erros_reconstrucao[y_test == 0], color='#2ecc71', bins=50, alpha=0.6, label='Normal', ax=axes[0])
sns.histplot(erros_reconstrucao[y_test == 1], color='#e74c3c', bins=50, alpha=0.6, label='Fraude', ax=axes[0])
axes[0].axvline(threshold, color='black', linestyle='--', linewidth=2, label='Limiar (Threshold)')
axes[0].set_yscale('log')
axes[0].set_title('Erro de Reconstrução FHE (Escala Log)', fontsize=12)
axes[0].set_xlabel('Erro Quadrático Médio (MSE)')
axes[0].set_ylabel('Frequência (Log)')
axes[0].legend()

# Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'],
            cbar=False, annot_kws={"size": 14})
axes[1].set_title('Matriz de Confusão (Agregação Segura)', fontsize=12)
axes[1].set_xlabel('Predito')
axes[1].set_ylabel('Real')

plt.tight_layout()
plt.show()

print("\nFim do processamento (Fase 3 Otimizada para Credit Card Fraud)!")
```

```
Dispositivo PyTorch: cuda

--- 1. Carregamento e Preparação dos Dados ---

Arquivo 'creditcard.csv' carregado com sucesso!
Realizando amostragem (100k linhas) para viabilidade do FHE...
Aplicando Engenharia de Features...
Dimensão de entrada (Features): 29

--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---

Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...
-> Contexto CKKS gerado com sucesso!

--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---

[Cliente 3] Treino e Criptografia concluídos em 4.02s.
[Cliente 2] Treino e Criptografia concluídos em 4.21s.
[Cliente 1] Treino e Criptografia concluídos em 4.68s.

--- 4. Agregação Federada CEGA (Servidor Central) ---

Servidor calculando operações sobre dados cifrados (FHE)...
-> Agregação matemática FHE finalizada em 0.00s.

--- 5. Descriptografia e Inferência ---

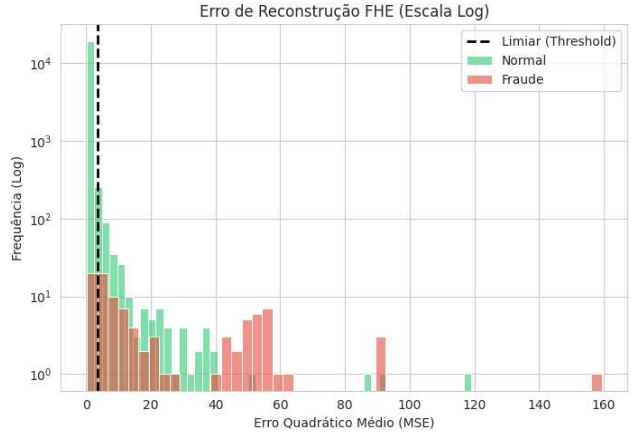
-> Modelo Global atualizado com os pesos agregados!

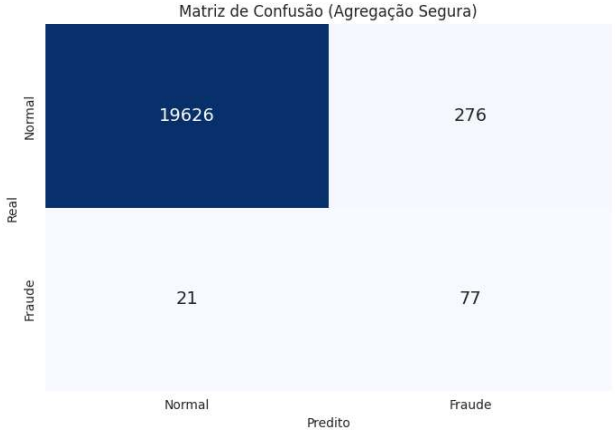
--- 5.1 Otimização de Limiar (Base de Validação) ---

Limiar de Anomalia Definido (P98 na Validação): 3.6548

--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---


>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):
Acurácia Geral      : 98.52%
Precisão (Precision): 21.81%
Revocação (Recall)  : 78.57%
F1-Score            : 34.15%
ROC-AUC             : 0.9551
Throughput de Infe. : 13742804.72 TPS
```






```
Fim do processamento (Fase 3 Otimizada para Credit Card Fraud)!
```

Nova base: Healthcare Fraud Detection

As células abaixo foram **adicionadas** ao teste, sem alterar as anteriores. Elas repetem a célula da base Credit Card com a **mesma configuração** (modelo, hiperparâmetros, sementes, split 60/20/20, limiar e métricas) na base `healthcare_fraud_detection.csv`, carregada direto do Google Drive (`/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/`). O rótulo de fraude é a coluna `Is_Fraud`.

1. A primeira célula registra os resultados da base Credit Card (célula acima) para o comparativo.
2. A segunda executa o teste na base healthcare.
3. A terceira compara as duas bases.

Engenharia de features da base healthcare (a mesma nos seis testes):

- removidos `Provider_ID` e `Claim_ID` (identificadores, como `nameOrig`/`nameDest` na PaySim) e `Claim_Submission_Date` (carimbo de tempo, como `Time` na Credit Card);
- `claim_amount_log` e `approved_amount_log` (log dos valores, como `amount_log`) e `approved_ratio` = valor aprovado ÷ valor solicitado (análoga ao `errorBalance` da PaySim);
- One-Hot das categóricas (`Patient_Gender`, `Diagnosis_Code`, `Procedure_Code`, `Insurance_Type`, `Provider_Specialty`, `Patient_State`, `Claim_Status`, `Visit_Type`), com ausentes na categoria `Desconhecido`;
- mediana para numéricas ausentes (`Prior_Visits_12m`).

```
# ======================================================================
# NOVA CÉLULA — registra os resultados da base Credit Card (célula acima)
# para o comparativo com a base Healthcare, feito na última célula.
# ======================================================================
import numpy as np
from sklearn.metrics import average_precision_score

comparativo_bases = {}
try:
    # A base Credit Card tem 29 features após a engenharia (V1..V28 + amount_log)
    if X.shape[1] != 29:
        raise NameError('os resultados em memória não são da base Credit Card')
    comparativo_bases['Credit Card'] = {
        'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
        'ROC-AUC': float(roc_auc), 'PR-AUC': float(average_precision_score(y_test, erros_reconstrucao)), 'Precisão': float(
        'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(len(X_test) / tempo_inferencia),
    }
    print("Resultados da base Credit Card registrados para o comparativo.")
except NameError as erro:
    print(f"Execute antes a célula da base Credit Card para incluí-la no comparativo ({erro}).")
```

```
Resultados da base Credit Card registrados para o comparativo.
```

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 3 na base Healthcare Fraud Detection (mesma configuração da célula Credit Card).
Script Otimizado para Teste da PoC (Fase 3): Criptografia Homomórfica com TenSEAL/CKKS
Dataset: Healthcare Fraud Detection (Kaggle) | Rótulo: Is_Fraud
Modelo: Autoencoder Tabular (PyTorch) + Aprendizado Federado (FedAvg) Paralelizado
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import concurrent.futures
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

try:
    import tenseal as ts
except ImportError:
    print("ERRO: TenSEAL não está instalado no seu ambiente.")
    print("Por favor, execute: pip install tenseal")
    exit()

# ========================================
# FUNÇÃO PARA FIXAR O DETERMINISMO
# ========================================
def set_seed(seed=42):
    import random
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        # Forçar operações determinísticas na GPU (pode reduzir performance)
```

```
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

# Aplicar a semente antes de qualquer coisa
set_seed(42)

# Configurações de estilo e dispositivo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 5)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")

print("\n--- 1. Carregamento e Preparação dos Dados (Healthcare Fraud Detection) ---\n")
caminho_csv = '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv'
# Cópia da base no repositório, usada somente quando o Google Drive não está disponível (execução local)
caminhos_alternativos = ['PoC/base_de_dados/healthcare_fraud_detection.csv', 'base_de_dados/healthcare_fraud_detection.csv

try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Fora do Google Colab: procurando a cópia da base no repositório.")

caminho = next((c for c in [caminho_csv] + caminhos_alternativos if os.path.exists(c)), None)
if caminho is None:
    raise FileNotFoundError(f"Base não encontrada: '{caminho_csv}'. Verifique se o Google Drive está montado.")
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (Is_Fraud = 1): {int(df['Is_Fraud'].sum())} ({df['Is_Fraud'].mean():.2%})")

# Sub-sample para FHE (mesma regra da base Credit Card; esta base tem 10 mil linhas, então nada muda)
if len(df) > 100000:
    print("Realizando amostragem (100k linhas) para viabilidade do FHE...")
    df_fraudes = df[df['Is_Fraud'] == 1]
    df_normais = df[df['Is_Fraud'] == 0].sample(n=(100000 - len(df_fraudes)), random_state=42)
    df = pd.concat([df_normais, df_fraudes]).sample(frac=1, random_state=42).reset_index(drop=True)

# Identificadores (Provider_ID, Claim_ID) não generalizam, como nameOrig/nameDest na PaySim, e a data de
# envio é um carimbo de tempo, como 'Time' na base Credit Card: as três colunas são removidas.
colunas_remover = ['Provider_ID', 'Claim_ID', 'Claim_Submission_Date']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
df = df.dropna(subset=['Is_Fraud'])
df['Is_Fraud'] = df['Is_Fraud'].astype(int)
print(f"As colunas {colunas_remover} foram removidas (identificadores e data).")

print("Aplicando Engenharia de Features...")
# Valores monetários com cauda longa -> log (como amount_log nas outras bases) e proporção aprovada
# pela operadora (análoga ao errorBalance da PaySim: sinistros fraudulentos têm aprovação bem menor)
df['approved_ratio'] = df['Approved_Amount'] / df['Claim_Amount'].replace(0, np.nan)
df['claim_amount_log'] = np.log1p(df['Claim_Amount'].clip(lower=0))
df['approved_amount_log'] = np.log1p(df['Approved_Amount'].clip(lower=0))
df = df.drop(columns=['Claim_Amount', 'Approved_Amount'])

# Categóricas -> One-Hot (Procedure_Code é um código, não uma quantidade); valor ausente vira 'Desconhecido'
colunas_categoricas = ['Patient_Gender', 'Diagnosis_Code', 'Procedure_Code', 'Insurance_Type',
                       'Provider_Specialty', 'Patient_State', 'Claim_Status', 'Visit_Type']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes (Prior_Visits_12m em 3,5% das linhas) -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")

# Separação
X = df.drop(columns=['Is_Fraud']).values.astype(np.float32)
y = df['Is_Fraud'].values.astype(np.int32)

# Correção de Data Leakage: Split Duplo (Treino, Validação, Teste)
X_temp, X_test_raw, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train_raw, X_val_raw, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp # 0.25 de 0.8 = 0.2
)

# Fit apenas no X_train, transform no X_train, X_val e X_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_val = scaler.transform(X_val_raw)
```

```
X_test = scaler.transform(X_test_raw)

input_dim = X_train.shape[1]
print(f"Dimensão de entrada (Features): {input_dim}")

class TabularAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(TabularAutoencoder, self).__init__()
        # Para base PCA de cartão de crédito (~29 features), uma compressão simples para FHE
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 14),
            nn.ReLU(),
            nn.Linear(14, 8),
            nn.ReLU()
        )
        # OMITIDO a Sigmoid() no decoder pois o StandardScaler gera valores fora de [0,1]
        self.decoder = nn.Sequential(
            nn.Linear(8, 14),
            nn.ReLU(),
            nn.Linear(14, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

print("\n--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---\n")
print("Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...")
# O grau do polinômio (8192) dita a segurança e a capacidade de packing (quantos pesos cabem em um cipher).
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.global_scale = 2**40
context.generate_galois_keys()
print("-> Contexto CKKS gerado com sucesso!")

print("\n--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---\n")
NUM_CLIENTES = 3

# Filtra apenas dados normais para treinar o Autoencoder
X_train_normal = X_train[y_train == 0]
tamanho_chunk = len(X_train_normal) // NUM_CLIENTES
clientes_X = [X_train_normal[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelo_global = TabularAutoencoder(input_dim).to(device)
criterion = nn.MSELoss()

def treinar_cliente_e_criptografar(cliente_id, dados_X, state_dict_global, epochs=3, batch_size=64):
    """Função que roda em paralelo: Treina, achata os pesos nativamente e os encripta."""
    inicio = time.time()

    # 1. Setup local
    modelo_local = TabularAutoencoder(input_dim).to(device)
    modelo_local.load_state_dict(state_dict_global)
    modelo_local.train()
    optimizer = optim.Adam(modelo_local.parameters(), lr=0.01)

    # Otimização: Pin_memory acelera transferência para GPU
    dataset = TensorDataset(torch.tensor(dados_X, dtype=torch.float32), torch.tensor(dados_X, dtype=torch.float32))

    # NOVO: Garantir que o DataLoader use a mesma seed para o shuffle
    generator = torch.Generator(device='cpu') # Generator para dataloader sempre no CPU
    generator.manual_seed(42 + cliente_id) # Seed única mas determinística por cliente

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=(device.type == 'cuda'),
        generator=generator # Aplica o generator determinístico
    )

    # 2. Treinamento
    for _ in range(epochs):
        for data, target in loader:
            data, target = data.to(device, non_blocking=True), target.to(device, non_blocking=True)
            optimizer.zero_grad()
            output = modelo_local(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()
```

```
    # 3. Extração Otimizada de Pesos (Vetorização C++ nativa do PyTorch)
    pesos_planos = parameters_to_vector(modelo_local.parameters()).detach().cpu().numpy().tolist()

    # 4. Criptografia Homomórfica
    pesos_cifrados = ts.ckks_vector(context, pesos_planos)

    tempo_total = time.time() - inicio
    print(f"[Cliente {cliente_id+1}] Treino e Criptografia concluídos em {tempo_total:.2f}s.")
    return pesos_cifrados

# Usando ThreadPoolExecutor para simular os clientes operando simultaneamente
pesos_criptografados_clientes = []
state_dict_atual = {k: v.cpu() for k, v in modelo_global.state_dict().items()}

with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_CLIENTES) as executor:
    futures = [
        executor.submit(treinar_cliente_e_criptografar, i, clientes_X[i], state_dict_atual)
        for i in range(NUM_CLIENTES)
    ]
    for future in concurrent.futures.as_completed(futures):
        pesos_criptografados_clientes.append(future.result())

print("\n--- 4. Agregação Federada CEGA (Servidor Central) ---\n")
print("Servidor calculando operações sobre dados cifrados (FHE)...")
inicio_agregracao = time.time()

# Soma homomórfica
soma_cifrada = pesos_criptografados_clientes[0]
for i in range(1, NUM_CLIENTES):
    soma_cifrada += pesos_criptografados_clientes[i]

# Média homomórfica
media_cifrada = soma_cifrada * (1.0 / NUM_CLIENTES)

tempo_agreg = time.time() - inicio_agregracao
print(f"-> Agregação matemática FHE finalizada em {tempo_agreg:.2f}s.")

print("\n--- 5. Descriptografia e Inferência ---\n")
# Cliente decifra e restaura a topologia
pesos_planos_agregados = media_cifrada.decrypt()

# Restauração otimizada dos pesos para o modelo global
tensor_pesos = torch.tensor(pesos_planos_agregados, dtype=torch.float32).to(device)
vector_to_parameters(tensor_pesos, modelo_global.parameters())
print("-> Modelo Global atualizado com os pesos agregados!")

print("\n--- 5.1 Otimização de Limiar (Base de Validação) ---\n")
modelo_global.eval()

with torch.no_grad():
    X_val_tensor = torch.tensor(X_val, dtype=torch.float32).to(device)
    reconstrucoes_val = modelo_global(X_val_tensor)
    erros_reconstrucao_val = torch.mean((X_val_tensor - reconstrucoes_val) ** 2, dim=1).cpu().numpy()

# Limiar de anomalia (Fraude) fixado no Percentil 98 USANDO APENAS A VALIDAÇÃO
threshold = np.percentile(erros_reconstrucao_val, 98)
print(f"Limiar de Anomalia Definido (P98 na Validação): {threshold:.4f}")

print("\n--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---\n")
inicio_inferencia = time.time()

with torch.no_grad():
    X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
    reconstrucoes = modelo_global(X_test_tensor)
    erros_reconstrucao = torch.mean((X_test_tensor - reconstrucoes) ** 2, dim=1).cpu().numpy()

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

# Aplicando o limiar descoberto na validação sobre os dados de teste
y_pred = (erros_reconstrucao > threshold).astype(int)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, erros_reconstrucao)

print("\n>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):")
print(f"Acurácia Geral      : {acc * 100:.2f}%")
print(f"Precisão (Precision): {prec * 100:.2f}%")
```

```
print(f"Revocação (Recall)  : {rec * 100:.2f}%")
print(f"F1-Score            : {f1 * 100:.2f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"Throughput de Infe. : {len(X_test)/tempo_inferencia:.2f} TPS")

# Plotagem de Resultados
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribuição de Erros
sns.histplot(erros_reconstrucao[y_test == 0], color='#2ecc71', bins=50, alpha=0.6, label='Normal', ax=axes[0])
sns.histplot(erros_reconstrucao[y_test == 1], color='#e74c3c', bins=50, alpha=0.6, label='Fraude', ax=axes[0])
axes[0].axvline(threshold, color='black', linestyle='--', linewidth=2, label='Limiar (Threshold)')
axes[0].set_yscale('log')
axes[0].set_title('Erro de Reconstrução FHE (Escala Log)', fontsize=12)
axes[0].set_xlabel('Erro Quadrático Médio (MSE)')
axes[0].set_ylabel('Frequência (Log)')
axes[0].legend()

# Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'],
            cbar=False, annot_kws={"size": 14})
axes[1].set_title('Matriz de Confusão (Agregação Segura)', fontsize=12)
axes[1].set_xlabel('Predito')
axes[1].set_ylabel('Real')

plt.tight_layout()
plt.show()

print("\nFim do processamento (Fase 3 Otimizada para Healthcare Fraud)!")

from sklearn.metrics import average_precision_score

comparativo_bases = globals().get('comparativo_bases', {})
comparativo_bases['Healthcare'] = {
    'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
    'ROC-AUC': float(roc_auc), 'PR-AUC': float(average_precision_score(y_test, erros_reconstrucao)), 'Precisão': float(prec
    'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(len(X_test) / tempo_inferencia),
}
print("Resultados da base Healthcare registrados para o comparativo.")
```

```
Dispositivo PyTorch: cuda

--- 1. Carregamento e Preparação dos Dados (Healthcare Fraud Detection) ---

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=
Arquivo '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv' carregado: 10000 linha
As colunas ['Provider_ID', 'Claim_ID', 'Claim_Submission_Date'] foram removidas (identificadores e data).
Aplicando Engenharia de Features...
Engenharia de features concluída: 48 features.
Dimensão de entrada (Features): 48

--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---

Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...
-> Contexto CKKS gerado com sucesso!

--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---

[Cliente 2] Treino e Criptografia concluídos em 0.38s.
[Cliente 1] Treino e Criptografia concluídos em 0.45s.
[Cliente 3] Treino e Criptografia concluídos em 0.48s.

--- 4. Agregação Federada CEGA (Servidor Central) ---

Servidor calculando operações sobre dados cifrados (FHE)...
-> Agregação matemática FHE finalizada em 0.00s.

--- 5. Descriptografia e Inferência ---

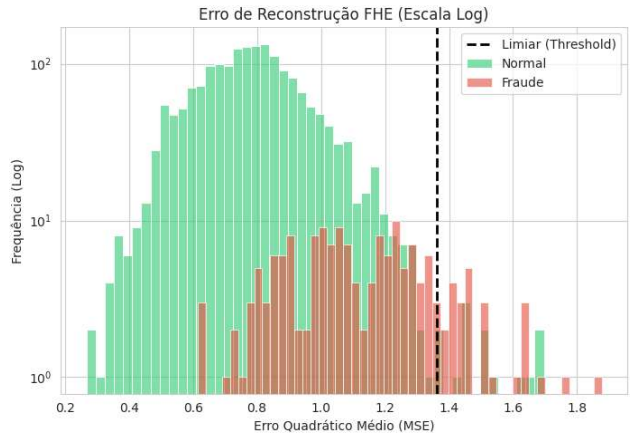
-> Modelo Global atualizado com os pesos agregados!

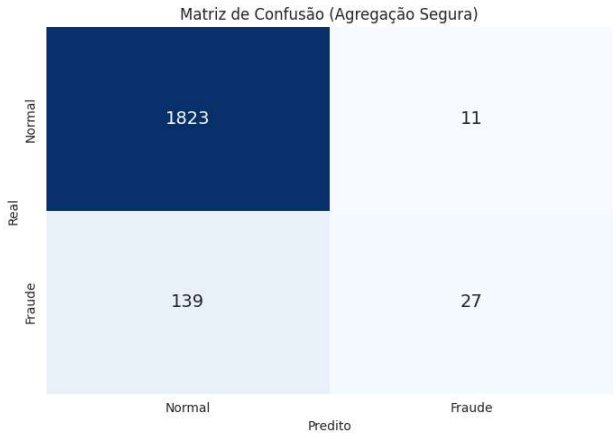
--- 5.1 Otimização de Limiar (Base de Validação) ---

Limiar de Anomalia Definido (P98 na Validação): 1.3624

--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---


>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):
Acurácia Geral      : 92.50%
Precisão (Precision): 71.05%
Revocação (Recall)  : 16.27%
F1-Score            : 26.47%
ROC-AUC             : 0.8724
Throughput de Infe. : 1899594.20 TPS
```






```
Fim do processamento (Fase 3 Otimizada para Healthcare Fraud)!
Resultados da base Healthcare registrados para o comparativo.
```

```
# ======================================================================
# NOVA CÉLULA — COMPARATIVO ENTRE AS BASES: Credit Card × Healthcare
# Teste 3 — Autoencoder federado com criptografia homomórfica (CKKS)
# ======================================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Cor fixa por base (paleta categórica validada para daltonismo): a mesma base tem sempre a mesma cor
CORES_BASES = {'Credit Card': '#2a78d6', 'Healthcare': '#eb6834', 'PaySim': '#1baf7a'}
METRICAS_COMP = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1-Score', 'Acurácia']
comparativo = pd.DataFrame(globals().get('comparativo_bases', {})).T
```

```
if comparativo.empty:
    print("Nenhum resultado registrado: execute as células das bases antes deste comparativo.")
else:
    # O PR-AUC de um classificador aleatório é igual à taxa de fraude. O PR-AUC normalizado desconta esse piso
    # (0 = aleatório, 1 = perfeito) e permite comparar bases com taxas de fraude diferentes.
    comparativo['PR-AUC normalizado'] = ((comparativo['PR-AUC'] - comparativo['Taxa de fraude'])
                                         / (1 - comparativo['Taxa de fraude']))

    print(">> CARACTERÍSTICAS DAS BASES")
    print(comparativo[['Transações', 'Taxa de fraude', 'Features']].to_string(formatters={
        'Transações': '{:,.0f}'.format, 'Taxa de fraude': '{:.2%}'.format, 'Features': '{:.0f}'.format}))

    print("\n>> MÉTRICAS NO CONJUNTO DE TESTE")
    print(comparativo[METRICAS_COMP + ['PR-AUC normalizado', 'Throughput (TPS)']].T.to_string(
        float_format=lambda v: f"{v:,.4f}"))

    if {'Credit Card', 'Healthcare'} <= set(comparativo.index):
        colunas_delta = METRICAS_COMP + ['PR-AUC normalizado']
        delta = comparativo.loc['Healthcare', colunas_delta] - comparativo.loc['Credit Card', colunas_delta]
        print("\n>> DIFERENÇA (Healthcare − Credit Card)")
        print(delta.map(lambda v: f"{v:+.4f}").to_string())

    print("\nObservação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. "
          "O PR-AUC depende dela (um classificador aleatório tem PR-AUC igual à taxa de fraude); por isso a "
          "comparação do PR-AUC entre bases deve usar o PR-AUC normalizado = (PR-AUC − taxa) / (1 − taxa).")

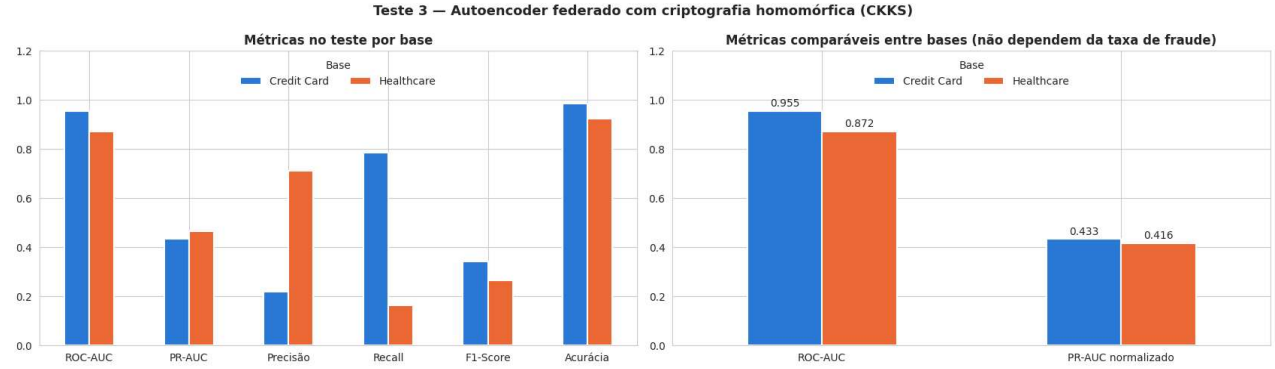
    fig, axes = plt.subplots(1, 2, figsize=(17, 5))
    cores = [CORES_BASES.get(b, '#7f7f7f') for b in comparativo.index]
    comparativo[METRICAS_COMP].T.plot.bar(ax=axes[0], rot=0, color=cores, edgecolor='white', linewidth=1.5)
    axes[0].set_ylim(0, 1.2)
    axes[0].set_title('Métricas no teste por base', fontweight='bold')
    axes[0].legend(title='Base', loc='upper center', ncol=len(comparativo), frameon=False)
    comparativo[['ROC-AUC', 'PR-AUC normalizado']].T.plot.bar(ax=axes[1], rot=0, color=cores,
                                                              edgecolor='white', linewidth=1.5)
    for barras in axes[1].containers:
        axes[1].bar_label(barras, fmt='%.3f', padding=2)
    axes[1].set_ylim(0, 1.2)
    axes[1].set_title('Métricas comparáveis entre bases (não dependem da taxa de fraude)', fontweight='bold')
    axes[1].legend(title='Base', loc='upper center', ncol=len(comparativo), frameon=False)
    plt.suptitle('Teste 3 — Autoencoder federado com criptografia homomórfica (CKKS)', fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

```
>> CARACTERÍSTICAS DAS BASES
            Transações Taxa de fraude Features
Credit Card    100,000          0.49%       29
Healthcare      10,000          8.29%       48

>> MÉTRICAS NO CONJUNTO DE TESTE
                       Credit Card     Healthcare
ROC-AUC                     0.9551         0.8724
PR-AUC                      0.4361         0.4640
Precisão                    0.2181         0.7105
Recall                      0.7857         0.1627
F1-Score                    0.3415         0.2647
Acurácia                    0.9851         0.9250
PR-AUC normalizado          0.4334         0.4156
Throughput (TPS)   13,742,804.7182 1,899,594.2029

>> DIFERENÇA (Healthcare - Credit Card)
ROC-AUC               -0.0827
PR-AUC                +0.0279
Precisão              +0.4924
Recall                -0.6231
F1-Score              -0.0768
Acurácia              -0.0601
PR-AUC normalizado    -0.0178

Observação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. O PR-AUC depende dela
```



Novas bases: Bank Account Fraud (NeurIPS 2022), Insurance Claims e Fraud Oracle

As células abaixo foram **adicionadas** ao teste, sem alterar as anteriores. Elas repetem a célula da base Credit Card com a **mesma configuração** (modelo, hiperparâmetros, sementes, split 60/20/20, limiar e métricas) em mais três bases.

Origem das bases:

- **Bank Account Fraud Dataset Suite (NeurIPS 2022)** — só o arquivo `Base.csv` (213 MB, 1 milhão de solicitações de abertura de conta, ~1,1% de fraude), baixado com `kagglehub` (já instalado no Colab; localmente, `pip install kagglehub`): `kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")`. Sem acesso ao Kaggle, é usada a cópia do repositório, `PoC/base_de_dados/Base.csv.zip`.
- **Insurance Claims** — `insurance_claims.csv` (1.000 sinistros de seguro de automóvel, 24,7% de fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/insurance_claims.csv`: a célula usa a cópia local quando o notebook roda a partir do repositório e, no Colab, lê o arquivo direto do GitHub.
- **Fraud Oracle** — `fraud_oracle.csv` (15.420 sinistros de seguro de veículos, 6,0% de fraude; rótulo `FraudFound_P`: 1 = fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/fraud_oracle.csv`, lida da mesma forma que a Insurance Claims.

1. A primeira célula executa o teste na base BAF.
2. A segunda executa o teste na base Insurance Claims.
3. A terceira executa o teste na base Fraud Oracle.
4. A quarta compara as cinco bases (Credit Card, Healthcare, BAF, Insurance Claims e Fraud Oracle). Para incluir as duas primeiras, execute antes a célula da base Credit Card, a célula que registra os seus resultados e a célula da base Healthcare.

Engenharia de features (a mesma nos seis testes):

- **BAF** (rótulo `fraud_bool`): removidos `month` (mês da solicitação, carimbo de tempo como `Time` na Credit Card) e `device_fraud_count` (constante na `Base.csv`); nas seis colunas em que valor negativo significa ausente (`prev_address_months_count`, `current_address_months_count`, `intended_balcon_amount`, `bank_months_count`,

`session_length_in_minutes`, `device_distinct_emails_8w`), criado um indicador `_ausente` e o valor ausente vai para a mediana; log de `intended_balcon_amount`, `days_since_request`, `zip_count_4w` e `proposed_credit_limit`; One-Hot de `payment_type`, `employment_status`, `housing_status`, `source` e `device_os`.

- **Insurance Claims** (rótulo `fraud_reported`, Y/N → 1/0): removidos `policy_number`, `insured_zip` e `incident_location` (identificadores, como `nameOrig`/`nameDest` na PaySim), `_c39` (coluna vazia) e `auto_model` (39 modelos para 1.000 linhas; `auto_make` é mantida); as datas `policy_bind_date` e `incident_date` viram `dias_apolice_ate_incidente`; log de `total_claim_amount`, `injury_claim`, `property_claim` e `vehicle_claim`; One-Hot das 16 categóricas, com `?` na categoria `Desconhecido` e `authorities_contacted` ausente (texto `None` no CSV) como `Nenhuma`.
- **Fraud Oracle** (rótulo `FraudFound_P`): removidos `PolicyNumber` e `RepNumber` (identificadores), `Year` (carimbo de tempo, como `Time`) e `PolicyType` (repete `VehicleCategory` + `BasePolicy`); `Month` e `MonthClaimed` viram `meses_ate_sinistro`; as nove colunas de faixas em texto (`VehiclePrice`, `Days_Policy_Accident`, `Days_Policy_Claim`, `PastNumberOfClaims`, `AgeOfVehicle`, `AgeOfPolicyHolder`, `NumberOfSuppliments`, `AddressChange_Claim`, `NumberOfCars`) viram números em ordem crescente; `Age` = 0 (inválido) vai para a mediana; One-Hot das 12 categóricas, com `0` em `DayOfWeekClaimed` como `Desconhecido`.

Amostragem do FHE no Teste 3: a regra da base Credit Card (100 mil linhas com todas as fraudes) é mantida quando as fraudes cabem em 10% da amostra. A BAF tem ~11 mil fraudes; manter todas levaria a amostra a ~11% de fraude (10 vezes a taxa real), então nela a amostra de 100 mil linhas é estratificada e preserva a taxa de 1,1%. A Insurance Claims (1.000 linhas) e a Fraud Oracle (15.420 linhas) são usadas inteiras.

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 3 na base Bank Account Fraud (NeurIPS 2022) — Base.csv (mesma configuração da célula Credit Card).
Script Otimizado para Teste da PoC (Fase 3): Criptografia Homomórfica com TenSEAL/CKKS
Dataset: sgpjesus/bank-account-fraud-dataset-neurips-2022, Base.csv (Kaggle, via kagglehub) | Rótulo: fraud_bool
Modelo: Autoencoder Tabular (PyTorch) + Aprendizado Federado (FedAvg) Paralelizado
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import concurrent.futures
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

try:
    import tenseal as ts
except ImportError:
    print("ERRO: TenSEAL não está instalado no seu ambiente.")
    print("Por favor, execute: pip install tenseal")
    exit()

# ========================================
# FUNÇÃO PARA FIXAR O DETERMINISMO
# ========================================
def set_seed(seed=42):
    import random
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        # Forçar operações determinísticas na GPU (pode reduzir performance)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

# Aplicar a semente antes de qualquer coisa
set_seed(42)

# Configurações de estilo e dispositivo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 5)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")

print("\n--- 1. Carregamento e Preparação dos Dados (Bank Account Fraud (NeurIPS 2022) — Base.csv) ---\n")
caminho = None
try:
    import kagglehub
    # Download latest version — apenas o arquivo Base.csv (213 MB) do Bank Account Fraud Dataset Suite
    path = kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")
    print("Path to dataset files:", path)
    caminho = path
except Exception as erro:
    print(f"Download pelo kagglehub indisponível ({type(erro).__name__}): procurando uma cópia local da base.")
if caminho is None or not os.path.exists(caminho):
    # Cópia no repositório (zip lido direto pelo pandas); executando da raiz do repositório ou de dentro de PoC/
    caminho = next((c for c in ['PoC/base_de_dados/Base.csv.zip', 'base_de_dados/Base.csv.zip',
                                'PoC/base_de_dados/Base.csv', 'base_de_dados/Base.csv'] if os.path.exists(c)), None)
if caminho is None:
    raise FileNotFoundError("Base.csv (Bank Account Fraud) não encontrada: verifique o acesso ao Kaggle (kagglehub) "
                            "ou copie o arquivo para PoC/base_de_dados/.")
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (fraud_bool = 1): {int(df['fraud_bool'].sum())} ({df['fraud_bool'].mean():.2%})")

# Sub-sample para FHE (100k linhas, mesma regra das outras bases: todas as fraudes + amostra de normais).
# Esta base tem ~11 mil fraudes (1,1%): acima de 10 mil fraudes (10% da amostra), a amostra de
# 100k é estratificada e preserva a taxa de fraude (manter todas deixaria ~11% de fraude na amostra).
if len(df) > 100000:
    print("Realizando amostragem (100k linhas) para viabilidade do FHE...")
    df_fraudes = df[df['fraud_bool'] == 1]
    if len(df_fraudes) <= 10000:
        df_normais = df[df['fraud_bool'] == 0].sample(n=(100000 - len(df_fraudes)), random_state=42)
        df = pd.concat([df_normais, df_fraudes]).sample(frac=1, random_state=42).reset_index(drop=True)
    else:
        df, _ = train_test_split(df, train_size=100000, stratify=df['fraud_bool'], random_state=42)
        df = df.reset_index(drop=True)
    print(f"Amostra: {len(df)} linhas | taxa de fraude {df['fraud_bool'].mean():.2%}")

# 'month' é o mês da solicitação (carimbo de tempo, como 'Time' na base Credit Card) e 'device_fraud_count' é
# constante (zero) na Base.csv: as duas colunas são removidas.
df = df.dropna(subset=['fraud_bool'])
df['fraud_bool'] = df['fraud_bool'].astype(int)
colunas_remover = ['month', 'device_fraud_count']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
print(f"As colunas {colunas_remover} foram removidas (tempo e coluna constante).")

print("Aplicando Engenharia de Features...")
# Na BAF, valores negativos nestas colunas codificam "ausente" (documentação do dataset): cada uma ganha um
# indicador de ausência e o valor ausente vai para a mediana (abaixo)
colunas_ausente_negativo = ['prev_address_months_count', 'current_address_months_count', 'intended_balcon_amount',
                            'bank_months_count', 'session_length_in_minutes', 'device_distinct_emails_8w']
for col in colunas_ausente_negativo:
    df[col + '_ausente'] = (df[col] < 0).astype(float)
    df[col] = df[col].where(df[col] >= 0)

# Contagens e valores com cauda longa -> log (como amount_log nas outras bases)
for col in ['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit']:
    df[col + '_log'] = np.log1p(df[col].clip(lower=0))
df = df.drop(columns=['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit'])

# Categóricas -> One-Hot; valor ausente vira 'Desconhecido'
colunas_categoricas = ['payment_type', 'employment_status', 'housing_status', 'source', 'device_os']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")

# Separação
X = df.drop(columns=['fraud_bool']).values.astype(np.float32)
y = df['fraud_bool'].values.astype(np.int32)

# Correção de Data Leakage: Split Duplo (Treino, Validação, Teste)
X_temp, X_test_raw, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train_raw, X_val_raw, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp # 0.25 de 0.8 = 0.2
```

```
)

# Fit apenas no X_train, transform no X_train, X_val e X_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_val = scaler.transform(X_val_raw)
X_test = scaler.transform(X_test_raw)

input_dim = X_train.shape[1]
print(f"Dimensão de entrada (Features): {input_dim}")

class TabularAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(TabularAutoencoder, self).__init__()
        # Para base PCA de cartão de crédito (~29 features), uma compressão simples para FHE
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 14),
            nn.ReLU(),
            nn.Linear(14, 8),
            nn.ReLU()
        )
        # OMITIDO a Sigmoid() no decoder pois o StandardScaler gera valores fora de [0,1]
        self.decoder = nn.Sequential(
            nn.Linear(8, 14),
            nn.ReLU(),
            nn.Linear(14, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

print("\n--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---\n")
print("Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...")
# O grau do polinômio (8192) dita a segurança e a capacidade de packing (quantos pesos cabem em um cipher).
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.global_scale = 2**40
context.generate_galois_keys()
print("-> Contexto CKKS gerado com sucesso!")

print("\n--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---\n")
NUM_CLIENTES = 3

# Filtra apenas dados normais para treinar o Autoencoder
X_train_normal = X_train[y_train == 0]
tamanho_chunk = len(X_train_normal) // NUM_CLIENTES
clientes_X = [X_train_normal[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelo_global = TabularAutoencoder(input_dim).to(device)
criterion = nn.MSELoss()

def treinar_cliente_e_criptografar(cliente_id, dados_X, state_dict_global, epochs=3, batch_size=64):
    """Função que roda em paralelo: Treina, achata os pesos nativamente e os encripta."""
    inicio = time.time()

    # 1. Setup local
    modelo_local = TabularAutoencoder(input_dim).to(device)
    modelo_local.load_state_dict(state_dict_global)
    modelo_local.train()
    optimizer = optim.Adam(modelo_local.parameters(), lr=0.01)

    # Otimização: Pin_memory acelera transferência para GPU
    dataset = TensorDataset(torch.tensor(dados_X, dtype=torch.float32), torch.tensor(dados_X, dtype=torch.float32))

    # NOVO: Garantir que o DataLoader use a mesma seed para o shuffle
    generator = torch.Generator(device='cpu') # Generator para dataloader sempre no CPU
    generator.manual_seed(42 + cliente_id) # Seed única mas determinística por cliente

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=(device.type == 'cuda'),
        generator=generator # Aplica o generator determinístico
    )

    # 2. Treinamento
    for _ in range(epochs):
        for data, target in loader:
```

```
            data, target = data.to(device, non_blocking=True), target.to(device, non_blocking=True)
            optimizer.zero_grad()
            output = modelo_local(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()

    # 3. Extração Otimizada de Pesos (Vetorização C++ nativa do PyTorch)
    pesos_planos = parameters_to_vector(modelo_local.parameters()).detach().cpu().numpy().tolist()

    # 4. Criptografia Homomórfica
    pesos_cifrados = ts.ckks_vector(context, pesos_planos)

    tempo_total = time.time() - inicio
    print(f"[Cliente {cliente_id+1}] Treino e Criptografia concluídos em {tempo_total:.2f}s.")
    return pesos_cifrados

# Usando ThreadPoolExecutor para simular os clientes operando simultaneamente
pesos_criptografados_clientes = []
state_dict_atual = {k: v.cpu() for k, v in modelo_global.state_dict().items()}

with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_CLIENTES) as executor:
    futures = [
        executor.submit(treinar_cliente_e_criptografar, i, clientes_X[i], state_dict_atual)
        for i in range(NUM_CLIENTES)
    ]
    for future in concurrent.futures.as_completed(futures):
        pesos_criptografados_clientes.append(future.result())

print("\n--- 4. Agregação Federada CEGA (Servidor Central) ---\n")
print("Servidor calculando operações sobre dados cifrados (FHE)...")
inicio_agregracao = time.time()

# Soma homomórfica
soma_cifrada = pesos_criptografados_clientes[0]
for i in range(1, NUM_CLIENTES):
    soma_cifrada += pesos_criptografados_clientes[i]

# Média homomórfica
media_cifrada = soma_cifrada * (1.0 / NUM_CLIENTES)

tempo_agreg = time.time() - inicio_agregracao
print(f"-> Agregação matemática FHE finalizada em {tempo_agreg:.2f}s.")

print("\n--- 5. Descriptografia e Inferência ---\n")
# Cliente decifra e restaura a topologia
pesos_planos_agregados = media_cifrada.decrypt()

# Restauração otimizada dos pesos para o modelo global
tensor_pesos = torch.tensor(pesos_planos_agregados, dtype=torch.float32).to(device)
vector_to_parameters(tensor_pesos, modelo_global.parameters())
print("-> Modelo Global atualizado com os pesos agregados!")

print("\n--- 5.1 Otimização de Limiar (Base de Validação) ---\n")
modelo_global.eval()

with torch.no_grad():
    X_val_tensor = torch.tensor(X_val, dtype=torch.float32).to(device)
    reconstrucoes_val = modelo_global(X_val_tensor)
    erros_reconstrucao_val = torch.mean((X_val_tensor - reconstrucoes_val) ** 2, dim=1).cpu().numpy()

# Limiar de anomalia (Fraude) fixado no Percentil 98 USANDO APENAS A VALIDAÇÃO
threshold = np.percentile(erros_reconstrucao_val, 98)
print(f"Limiar de Anomalia Definido (P98 na Validação): {threshold:.4f}")

print("\n--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---\n")
inicio_inferencia = time.time()

with torch.no_grad():
    X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
    reconstrucoes = modelo_global(X_test_tensor)
    erros_reconstrucao = torch.mean((X_test_tensor - reconstrucoes) ** 2, dim=1).cpu().numpy()

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

# Aplicando o limiar descoberto na validação sobre os dados de teste
y_pred = (erros_reconstrucao > threshold).astype(int)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
```

```
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, erros_reconstrucao)

print("\n>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):")
print(f"Acurácia Geral      : {acc * 100:.2f}%")
print(f"Precisão (Precision): {prec * 100:.2f}%")
print(f"Revocação (Recall)  : {rec * 100:.2f}%")
print(f"F1-Score            : {f1 * 100:.2f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"Throughput de Infe. : {len(X_test)/tempo_inferencia:.2f} TPS")

# Plotagem de Resultados
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribuição de Erros
sns.histplot(erros_reconstrucao[y_test == 0], color='#2ecc71', bins=50, alpha=0.6, label='Normal', ax=axes[0])
sns.histplot(erros_reconstrucao[y_test == 1], color='#e74c3c', bins=50, alpha=0.6, label='Fraude', ax=axes[0])
axes[0].axvline(threshold, color='black', linestyle='--', linewidth=2, label='Limiar (Threshold)')
axes[0].set_yscale('log')
axes[0].set_title('Erro de Reconstrução FHE (Escala Log)', fontsize=12)
axes[0].set_xlabel('Erro Quadrático Médio (MSE)')
axes[0].set_ylabel('Frequência (Log)')
axes[0].legend()

# Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'],
            cbar=False, annot_kws={"size": 14})
axes[1].set_title('Matriz de Confusão (Agregação Segura)', fontsize=12)
axes[1].set_xlabel('Predito')
axes[1].set_ylabel('Real')

plt.tight_layout()
plt.show()

print("\nFim do processamento (Fase 3 Otimizada para BAF Fraud)!")

from sklearn.metrics import average_precision_score

comparativo_bases = globals().get('comparativo_bases', {})
comparativo_bases['BAF'] = {
    'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
    'ROC-AUC': float(roc_auc), 'PR-AUC': float(average_precision_score(y_test, erros_reconstrucao)), 'Precisão': float(prec
    'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(len(X_test) / tempo_inferencia),
}
print("Resultados da base BAF registrados para o comparativo.")
```

```
Dispositivo PyTorch: cuda

--- 1. Carregamento e Preparação dos Dados (Bank Account Fraud (NeurIPS 2022) — Base.csv) ---

Using Colab cache for faster access to the 'bank-account-fraud-dataset-neurips-2022' dataset.
Path to dataset files: /kaggle/input/bank-account-fraud-dataset-neurips-2022/Base.csv
Arquivo '/kaggle/input/bank-account-fraud-dataset-neurips-2022/Base.csv' carregado: 1000000 linhas, 32 colunas | fraudes (
Realizando amostragem (100k linhas) para viabilidade do FHE...
Amostra: 100000 linhas | taxa de fraude 1.10%
As colunas ['month', 'device_fraud_count'] foram removidas (tempo e coluna constante).
Aplicando Engenharia de Features...
Engenharia de features concluída: 51 features.
Dimensão de entrada (Features): 51

--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---

Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...
-> Contexto CKKS gerado com sucesso!

--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---

[Cliente 1] Treino e Criptografia concluídos em 4.45s.
[Cliente 2] Treino e Criptografia concluídos em 4.58s.
[Cliente 3] Treino e Criptografia concluídos em 4.97s.

--- 4. Agregação Federada CEGA (Servidor Central) ---

Servidor calculando operações sobre dados cifrados (FHE)...
-> Agregação matemática FHE finalizada em 0.00s.

--- 5. Descriptografia e Inferência ---

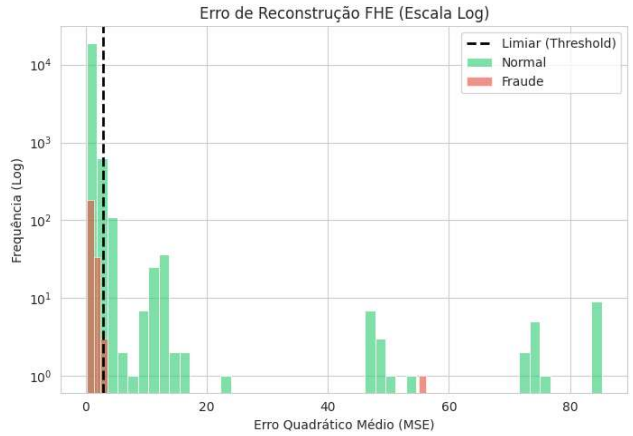
-> Modelo Global atualizado com os pesos agregados!

--- 5.1 Otimização de Limiar (Base de Validação) ---

Limiar de Anomalia Definido (P98 na Validação): 2.8870

--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---


>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):
Acurácia Geral      : 96.81%
Precisão (Precision): 0.71%
Revocação (Recall)  : 1.36%
F1-Score            : 0.93%
ROC-AUC             : 0.5035
Throughput de Infe. : 9203080.64 TPS
```






```
Fim do processamento (Fase 3 Otimizada para BAF Fraud)!
Resultados da base BAF registrados para o comparativo.
```

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 3 na base Insurance Claims (sinistros de seguro de automóvel) (mesma configuração da célula Credit Carc
Script Otimizado para Teste da PoC (Fase 3): Criptografia Homomórfica com TenSEAL/CKKS
Dataset: insurance_claims.csv (repositório GitHub, PoC/base_de_dados/) | Rótulo: fraud_reported
Modelo: Autoencoder Tabular (PyTorch) + Aprendizado Federado (FedAvg) Paralelizado
"""

import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import concurrent.futures
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

try:
    import tenseal as ts
except ImportError:
    print("ERRO: TenSEAL não está instalado no seu ambiente.")
    print("Por favor, execute: pip install tenseal")
    exit()

# ========================================
# FUNÇÃO PARA FIXAR O DETERMINISMO
# ========================================
def set_seed(seed=42):
    import random
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        # Forçar operações determinísticas na GPU (pode reduzir performance)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

# Aplicar a semente antes de qualquer coisa
set_seed(42)

# Configurações de estilo e dispositivo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 5)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")

print("\n--- 1. Carregamento e Preparação dos Dados (Insurance Claims (sinistros de seguro de automóvel)) ---\n")
# Base no repositório GitHub do projeto (PoC/base_de_dados/insurance_claims.csv): usa a cópia local quando o
# notebook roda a partir do repositório; no Colab, lê o arquivo direto do GitHub
URL_GITHUB = 'https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/insurance_claims.csv'
caminho = next((c for c in ['PoC/base_de_dados/insurance_claims.csv', 'base_de_dados/insurance_claims.csv']
                if os.path.exists(c)), URL_GITHUB)
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (fraud_reported = Y): {int((df['fraud_reported'] == 'Y').sum())} ({(df['fraud_reported'] == 'Y').mean():.2%

# Sub-sample para FHE: só acima de 100k linhas (mesma regra das outras bases); esta base tem 1.000 linhas e é usada inteira

# Rótulo Y/N -> 1/0. Identificadores e texto único por linha (policy_number, insured_zip, incident_location) não
# generalizam, como nameOrig/nameDest na PaySim; _c39 é uma coluna vazia; auto_model tem 39 modelos para 1.000 linhas
# (auto_make é mantida). As duas datas viram um único intervalo: dias entre a contratação da apólice e o incidente.
df['fraud_reported'] = df['fraud_reported'].astype(str).str.strip().str.upper().map({'Y': 1, 'N': 0})
df = df.dropna(subset=['fraud_reported'])
df['fraud_reported'] = df['fraud_reported'].astype(int)
df['dias_apolice_ate_incidente'] = (pd.to_datetime(df['incident_date'], errors='coerce')
                                    - pd.to_datetime(df['policy_bind_date'], errors='coerce')).dt.days
colunas_remover = ['policy_number', 'insured_zip', 'incident_location', '_c39', 'auto_model',
                   'policy_bind_date', 'incident_date']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
print(f"As colunas {colunas_remover} foram removidas (identificadores, coluna vazia, alta cardinalidade e datas).")

print("Aplicando Engenharia de Features...")
# Valores do sinistro com cauda longa -> log (como amount_log nas outras bases)
for col in ['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim']:
    df[col + '_log'] = np.log1p(df[col].clip(lower=0))
df = df.drop(columns=['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim'])

# Categóricas -> One-Hot. '?' marca valor desconhecido; em authorities_contacted, o texto 'None' (nenhuma autoridade
# acionada) é lido pelo pandas como ausente e vira 'Nenhuma'
```

```
df['authorities_contacted'] = df['authorities_contacted'].fillna('Nenhuma')
colunas_categoricas = ['policy_state', 'policy_csl', 'insured_sex', 'insured_education_level', 'insured_occupation',
                       'insured_hobbies', 'insured_relationship', 'incident_type', 'collision_type', 'incident_severity',
                       'authorities_contacted', 'incident_state', 'incident_city', 'property_damage',
                       'police_report_available', 'auto_make']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('?', 'Desconhecido')
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")

# Separação
X = df.drop(columns=['fraud_reported']).values.astype(np.float32)
y = df['fraud_reported'].values.astype(np.int32)

# Correção de Data Leakage: Split Duplo (Treino, Validação, Teste)
X_temp, X_test_raw, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train_raw, X_val_raw, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp # 0.25 de 0.8 = 0.2
)

# Fit apenas no X_train, transform no X_train, X_val e X_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_val = scaler.transform(X_val_raw)
X_test = scaler.transform(X_test_raw)

input_dim = X_train.shape[1]
print(f"Dimensão de entrada (Features): {input_dim}")

class TabularAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(TabularAutoencoder, self).__init__()
        # Para base PCA de cartão de crédito (~29 features), uma compressão simples para FHE
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 14),
            nn.ReLU(),
            nn.Linear(14, 8),
            nn.ReLU()
        )
        # OMITIDO a Sigmoid() no decoder pois o StandardScaler gera valores fora de [0,1]
        self.decoder = nn.Sequential(
            nn.Linear(8, 14),
            nn.ReLU(),
            nn.Linear(14, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

print("\n--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---\n")
print("Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...")
# O grau do polinômio (8192) dita a segurança e a capacidade de packing (quantos pesos cabem em um cipher).
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.global_scale = 2**40
context.generate_galois_keys()
print("-> Contexto CKKS gerado com sucesso!")

print("\n--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---\n")
NUM_CLIENTES = 3

# Filtra apenas dados normais para treinar o Autoencoder
X_train_normal = X_train[y_train == 0]
tamanho_chunk = len(X_train_normal) // NUM_CLIENTES
clientes_X = [X_train_normal[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelo_global = TabularAutoencoder(input_dim).to(device)
criterion = nn.MSELoss()

def treinar_cliente_e_criptografar(cliente_id, dados_X, state_dict_global, epochs=3, batch_size=64):
    """Função que roda em paralelo: Treina, achata os pesos nativamente e os encripta."""
    inicio = time.time()
```

```
    # 1. Setup local
    modelo_local = TabularAutoencoder(input_dim).to(device)
    modelo_local.load_state_dict(state_dict_global)
    modelo_local.train()
    optimizer = optim.Adam(modelo_local.parameters(), lr=0.01)

    # Otimização: Pin_memory acelera transferência para GPU
    dataset = TensorDataset(torch.tensor(dados_X, dtype=torch.float32), torch.tensor(dados_X, dtype=torch.float32))

    # NOVO: Garantir que o DataLoader use a mesma seed para o shuffle
    generator = torch.Generator(device='cpu') # Generator para dataloader sempre no CPU
    generator.manual_seed(42 + cliente_id) # Seed única mas determinística por cliente

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=(device.type == 'cuda'),
        generator=generator # Aplica o generator determinístico
    )

    # 2. Treinamento
    for _ in range(epochs):
        for data, target in loader:
            data, target = data.to(device, non_blocking=True), target.to(device, non_blocking=True)
            optimizer.zero_grad()
            output = modelo_local(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()

    # 3. Extração Otimizada de Pesos (Vetorização C++ nativa do PyTorch)
    pesos_planos = parameters_to_vector(modelo_local.parameters()).detach().cpu().numpy().tolist()

    # 4. Criptografia Homomórfica
    pesos_cifrados = ts.ckks_vector(context, pesos_planos)

    tempo_total = time.time() - inicio
    print(f"[Cliente {cliente_id+1}] Treino e Criptografia concluídos em {tempo_total:.2f}s.")
    return pesos_cifrados

# Usando ThreadPoolExecutor para simular os clientes operando simultaneamente
pesos_criptografados_clientes = []
state_dict_atual = {k: v.cpu() for k, v in modelo_global.state_dict().items()}

with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_CLIENTES) as executor:
    futures = [
        executor.submit(treinar_cliente_e_criptografar, i, clientes_X[i], state_dict_atual)
        for i in range(NUM_CLIENTES)
    ]
    for future in concurrent.futures.as_completed(futures):
        pesos_criptografados_clientes.append(future.result())

print("\n--- 4. Agregação Federada CEGA (Servidor Central) ---\n")
print("Servidor calculando operações sobre dados cifrados (FHE)...")
inicio_agregracao = time.time()

# Soma homomórfica
soma_cifrada = pesos_criptografados_clientes[0]
for i in range(1, NUM_CLIENTES):
    soma_cifrada += pesos_criptografados_clientes[i]

# Média homomórfica
media_cifrada = soma_cifrada * (1.0 / NUM_CLIENTES)

tempo_agreg = time.time() - inicio_agregracao
print(f"-> Agregação matemática FHE finalizada em {tempo_agreg:.2f}s.")

print("\n--- 5. Descriptografia e Inferência ---\n")
# Cliente decifra e restaura a topologia
pesos_planos_agregados = media_cifrada.decrypt()

# Restauração otimizada dos pesos para o modelo global
tensor_pesos = torch.tensor(pesos_planos_agregados, dtype=torch.float32).to(device)
vector_to_parameters(tensor_pesos, modelo_global.parameters())
print("-> Modelo Global atualizado com os pesos agregados!")

print("\n--- 5.1 Otimização de Limiar (Base de Validação) ---\n")
modelo_global.eval()

with torch.no_grad():
```

```
    X_val_tensor = torch.tensor(X_val, dtype=torch.float32).to(device)
    reconstrucoes_val = modelo_global(X_val_tensor)
    erros_reconstrucao_val = torch.mean((X_val_tensor - reconstrucoes_val) ** 2, dim=1).cpu().numpy()

# Limiar de anomalia (Fraude) fixado no Percentil 98 USANDO APENAS A VALIDAÇÃO
threshold = np.percentile(erros_reconstrucao_val, 98)
print(f"Limiar de Anomalia Definido (P98 na Validação): {threshold:.4f}")

print("\n--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---\n")
inicio_inferencia = time.time()

with torch.no_grad():
    X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
    reconstrucoes = modelo_global(X_test_tensor)
    erros_reconstrucao = torch.mean((X_test_tensor - reconstrucoes) ** 2, dim=1).cpu().numpy()

fim_inferencia = time.time()
tempo_inferencia = fim_inferencia - inicio_inferencia

# Aplicando o limiar descoberto na validação sobre os dados de teste
y_pred = (erros_reconstrucao > threshold).astype(int)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
roc_auc = roc_auc_score(y_test, erros_reconstrucao)

print("\n>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):")
print(f"Acurácia Geral      : {acc * 100:.2f}%")
print(f"Precisão (Precision): {prec * 100:.2f}%")
print(f"Revocação (Recall)  : {rec * 100:.2f}%")
print(f"F1-Score            : {f1 * 100:.2f}%")
print(f"ROC-AUC             : {roc_auc:.4f}")
print(f"Throughput de Infe. : {len(X_test)/tempo_inferencia:.2f} TPS")

# Plotagem de Resultados
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribuição de Erros
sns.histplot(erros_reconstrucao[y_test == 0], color='#2ecc71', bins=50, alpha=0.6, label='Normal', ax=axes[0])
sns.histplot(erros_reconstrucao[y_test == 1], color='#e74c3c', bins=50, alpha=0.6, label='Fraude', ax=axes[0])
axes[0].axvline(threshold, color='black', linestyle='--', linewidth=2, label='Limiar (Threshold)')
axes[0].set_yscale('log')
axes[0].set_title('Erro de Reconstrução FHE (Escala Log)', fontsize=12)
axes[0].set_xlabel('Erro Quadrático Médio (MSE)')
axes[0].set_ylabel('Frequência (Log)')
axes[0].legend()

# Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1],
            xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'],
            cbar=False, annot_kws={"size": 14})
axes[1].set_title('Matriz de Confusão (Agregação Segura)', fontsize=12)
axes[1].set_xlabel('Predito')
axes[1].set_ylabel('Real')

plt.tight_layout()
plt.show()

print("\nFim do processamento (Fase 3 Otimizada para Insurance Claims Fraud)!")

from sklearn.metrics import average_precision_score

comparativo_bases = globals().get('comparativo_bases', {})
comparativo_bases['Insurance Claims'] = {
    'Transações': int(len(y)), 'Taxa de fraude': float(np.mean(y)), 'Features': int(X.shape[1]),
    'ROC-AUC': float(roc_auc), 'PR-AUC': float(average_precision_score(y_test, erros_reconstrucao)), 'Precisão': float(prec
    'F1-Score': float(f1), 'Acurácia': float(acc), 'Throughput (TPS)': float(len(X_test) / tempo_inferencia),
}
print("Resultados da base Insurance Claims registrados para o comparativo.")
```

```
Dispositivo PyTorch: cuda

--- 1. Carregamento e Preparação dos Dados (Insurance Claims (sinistros de seguro de automóvel)) ---

Arquivo 'https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/insurance_claims.csv' carre
As colunas ['policy_number', 'insured_zip', 'incident_location', '_c39', 'auto_model', 'policy_bind_date', 'incident_date'
Aplicando Engenharia de Features...
Engenharia de features concluída: 107 features.
Dimensão de entrada (Features): 107

--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---

Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...
-> Contexto CKKS gerado com sucesso!

--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---

[Cliente 2] Treino e Criptografia concluídos em 0.07s.
[Cliente 3] Treino e Criptografia concluídos em 0.08s.
[Cliente 1] Treino e Criptografia concluídos em 0.08s.

--- 4. Agregação Federada CEGA (Servidor Central) ---

Servidor calculando operações sobre dados cifrados (FHE)...
-> Agregação matemática FHE finalizada em 0.00s.

--- 5. Descriptografia e Inferência ---

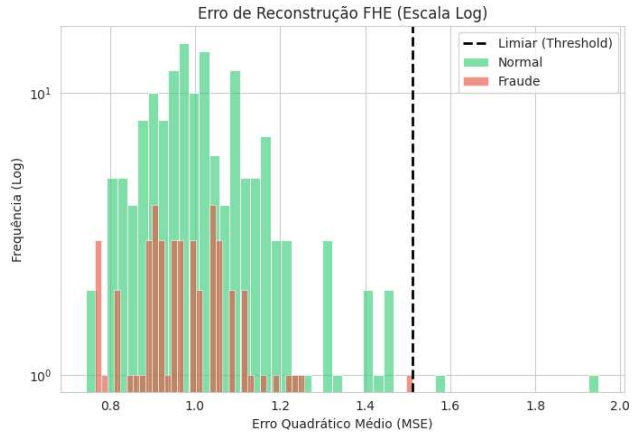
-> Modelo Global atualizado com os pesos agregados!

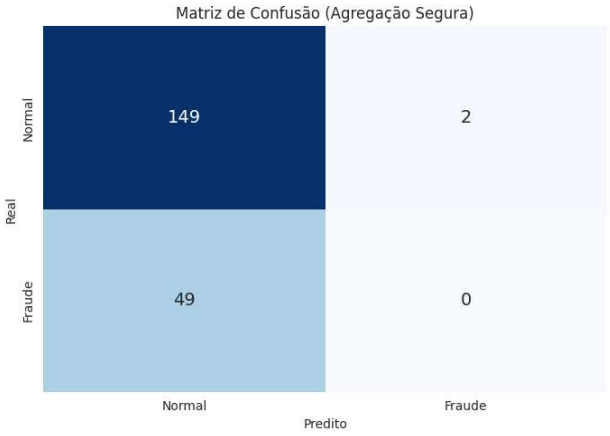
--- 5.1 Otimização de Limiar (Base de Validação) ---

Limiar de Anomalia Definido (P98 na Validação): 1.5121

--- 5.2 AVALIAÇÃO FINAL (Base de Teste) ---


>> MÉTRICAS DO MODELO COM CRIPTOGRAFIA FHE (CKKS):
Acurácia Geral      : 74.50%
Precisão (Precision): 0.00%
Revocação (Recall)  : 0.00%
F1-Score            : 0.00%
ROC-AUC             : 0.4288
Throughput de Infe. : 230077.02 TPS
```






```
Fim do processamento (Fase 3 Otimizada para Insurance Claims Fraud)!
Resultados da base Insurance Claims registrados para o comparativo.
```

```
# -*- coding: utf-8 -*-
"""
NOVA CÉLULA — Teste 3 na base Fraud Oracle (fraude em sinistros de seguro de veículos) (mesma configuração da célula Cred
Script Otimizado para Teste da PoC (Fase 3): Criptografia Homomórfica com TenSEAL/CKKS
Dataset: fraud_oracle.csv (repositório GitHub, PoC/base_de_dados/) | Rótulo: FraudFound_P
Modelo: Autoencoder Tabular (PyTorch) + Aprendizado Federado (FedAvg) Paralelizado
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import time
import concurrent.futures
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torch.nn.utils import parameters_to_vector, vector_to_parameters

try:
    import tenseal as ts
except ImportError:
    print("ERRO: TenSEAL não está instalado no seu ambiente.")
    print("Por favor, execute: pip install tenseal")
    exit()

# ========================================
# FUNÇÃO PARA FIXAR O DETERMINISMO
# ========================================
def set_seed(seed=42):
    import random
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        # Forçar operações determinísticas na GPU (pode reduzir performance)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

# Aplicar a semente antes de qualquer coisa
set_seed(42)

# Configurações de estilo e dispositivo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 5)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo PyTorch: {device}")

print("\n--- 1. Carregamento e Preparação dos Dados (Fraud Oracle (fraude em sinistros de seguro de veículos)) ---\n")
# Base no repositório GitHub do projeto (PoC/base_de_dados/fraud_oracle.csv): usa a cópia local quando o
# notebook roda a partir do repositório; no Colab, lê o arquivo direto do GitHub
URL_GITHUB = 'https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/fraud_oracle.csv'
caminho = next((c for c in ['PoC/base_de_dados/fraud_oracle.csv', 'base_de_dados/fraud_oracle.csv']
                if os.path.exists(c)), URL_GITHUB)
df = pd.read_csv(caminho)
print(f"Arquivo '{caminho}' carregado: {df.shape[0]} linhas, {df.shape[1]} colunas | "
      f"fraudes (FraudFound_P = 1): {int(df['FraudFound_P'].sum())} ({df['FraudFound_P'].mean():.2%})")

# Sub-sample para FHE: só acima de 100k linhas (mesma regra das outras bases); esta base tem 15.420 linhas e é usada inte

df = df.dropna(subset=['FraudFound_P'])
df['FraudFound_P'] = df['FraudFound_P'].astype(int)
# PolicyNumber (sequencial) e RepNumber (representante) são identificadores, como nameOrig/nameDest na PaySim;
# Year é carimbo de tempo, como 'Time' na base Credit Card; PolicyType repete VehicleCategory + BasePolicy.
# Month e MonthClaimed viram um único intervalo: meses entre o acidente e a abertura do sinistro.
MESES = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11,
df['meses_ate_sinistro'] = (df['MonthClaimed'].map(MESES) - df['Month'].map(MESES)) % 12
colunas_remover = ['PolicyNumber', 'RepNumber', 'Year', 'PolicyType', 'Month', 'MonthClaimed']
df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
print(f"As colunas {colunas_remover} foram removidas (identificadores, ano, coluna redundante e meses).")

print("Aplicando Engenharia de Features...")
# Faixas de texto -> números em ordem crescente (preserva a ordem e evita dezenas de colunas One-Hot)
FAIXAS_ORDINAIS = {
    'VehiclePrice': ['less than 20000', '20000 to 29000', '30000 to 39000', '40000 to 59000', '60000 to 69000', 'more tha
    'Days_Policy_Accident': ['none', '1 to 7', '8 to 15', '15 to 30', 'more than 30'],
    'Days_Policy_Claim': ['none', '8 to 15', '15 to 30', 'more than 30'],
    'PastNumberOfClaims': ['none', '1', '2 to 4', 'more than 4'],
    'AgeOfVehicle': ['new', '2 years', '3 years', '4 years', '5 years', '6 years', '7 years', 'more than 7'],
    'AgeOfPolicyHolder': ['16 to 17', '18 to 20', '21 to 25', '26 to 30', '31 to 35', '36 to 40', '41 to 50', '51 to 65',
    'NumberOfSuppliments': ['none', '1 to 2', '3 to 5', 'more than 5'],
    'AddressChange_Claim': ['under 6 months', '1 year', '2 to 3 years', '4 to 8 years', 'no change'],
    'NumberOfCars': ['1 vehicle', '2 vehicles', '3 to 4', '5 to 8', 'more than 8'],
```

```
}
for col, ordem in FAIXAS_ORDINAIS.items():
    df[col] = df[col].astype(str).str.strip().map({faixa: i for i, faixa in enumerate(ordem)})
# Age = 0 é um valor inválido na base (o motorista mais novo está na faixa 16 a 17 anos): tratado como ausente
df['Age'] = df['Age'].where(df['Age'] > 0)

# Categóricas -> One-Hot ('0' em DayOfWeekClaimed é valor desconhecido)
colunas_categoricas = ['DayOfWeek', 'DayOfWeekClaimed', 'Make', 'AccidentArea', 'Sex', 'MaritalStatus', 'Fault',
                       'VehicleCategory', 'PoliceReportFiled', 'WitnessPresent', 'AgentType', 'BasePolicy']
for col in colunas_categoricas:
    df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('0', 'Desconhecido')
df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

# Numéricas ausentes (faixas fora da lista, Age = 0, mês desconhecido) -> mediana
for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())
print(f"Engenharia de features concluída: {df.shape[1] - 1} features.")

# Separação
X = df.drop(columns=['FraudFound_P']).values.astype(np.float32)
y = df['FraudFound_P'].values.astype(np.int32)

# Correção de Data Leakage: Split Duplo (Treino, Validação, Teste)
X_temp, X_test_raw, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train_raw, X_val_raw, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp # 0.25 de 0.8 = 0.2
)

# Fit apenas no X_train, transform no X_train, X_val e X_test
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train_raw)
X_val = scaler.transform(X_val_raw)
X_test = scaler.transform(X_test_raw)

input_dim = X_train.shape[1]
print(f"Dimensão de entrada (Features): {input_dim}")

class TabularAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(TabularAutoencoder, self).__init__()
        # Para base PCA de cartão de crédito (~29 features), uma compressão simples para FHE
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 14),
            nn.ReLU(),
            nn.Linear(14, 8),
            nn.ReLU()
        )
        # OMITIDO a Sigmoid() no decoder pois o StandardScaler gera valores fora de [0,1]
        self.decoder = nn.Sequential(
            nn.Linear(8, 14),
            nn.ReLU(),
            nn.Linear(14, input_dim)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

print("\n--- 2. Setup de Criptografia Homomórfica (TenSEAL/CKKS) ---\n")
print("Gerando chaves criptográficas FHE (Isso pode levar alguns segundos)...")
# O grau do polinômio (8192) dita a segurança e a capacidade de packing (quantos pesos cabem em um cipher).
context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.global_scale = 2**40
context.generate_galois_keys()
print("-> Contexto CKKS gerado com sucesso!")

print("\n--- 3. Aprendizado Federado Paralelizado (Treinamento Local) ---\n")
NUM_CLIENTES = 3

# Filtra apenas dados normais para treinar o Autoencoder
X_train_normal = X_train[y_train == 0]
tamanho_chunk = len(X_train_normal) // NUM_CLIENTES
clientes_X = [X_train_normal[i * tamanho_chunk : (i + 1) * tamanho_chunk] for i in range(NUM_CLIENTES)]

modelo_global = TabularAutoencoder(input_dim).to(device)
criterion = nn.MSELoss()
```

```
def treinar_cliente_e_criptografar(cliente_id, dados_X, state_dict_global, epochs=3, batch_size=64):
    """Função que roda em paralelo: Treina, achata os pesos nativamente e os encripta."""
    inicio = time.time()

    # 1. Setup local
    modelo_local = TabularAutoencoder(input_dim).to(device)
    modelo_local.load_state_dict(state_dict_global)
    modelo_local.train()
    optimizer = optim.Adam(modelo_local.parameters(), lr=0.01)

    # Otimização: Pin_memory acelera transferência para GPU
    dataset = TensorDataset(torch.tensor(dados_X, dtype=torch.float32), torch.tensor(dados_X, dtype=torch.float32))

    # NOVO: Garantir que o DataLoader use a mesma seed para o shuffle
    generator = torch.Generator(device='cpu') # Generator para dataloader sempre no CPU
    generator.manual_seed(42 + cliente_id) # Seed única mas determinística por cliente

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=(device.type == 'cuda'),
        generator=generator # Aplica o generator determinístico
    )

    # 2. Treinamento
    for _ in range(epochs):
        for data, target in loader:
            data, target = data.to(device, non_blocking=True), target.to(device, non_blocking=True)
            optimizer.zero_grad()
            output = modelo_local(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()

    # 3. Extração Otimizada de Pesos (Vetorização C++ nativa do PyTorch)
    pesos_planos = parameters_to_vector(modelo_local.parameters()).detach().cpu().numpy().tolist()

    # 4. Criptografia Homomórfica
    pesos_cifrados = ts.ckks_vector(context, pesos_planos)

    tempo_total = time.time() - inicio
    print(f"[Cliente {cliente_id+1}] Treino e Criptografia concluídos em {tempo_total:.2f}s.")
    return pesos_cifrados

# Usando ThreadPoolExecutor para simular os clientes operando simultaneamente
pesos_criptografados_clientes = []
state_dict_atual = {k: v.cpu() for k, v in modelo_global.state_dict().items()}

with concurrent.futures.ThreadPoolExecutor(max_workers=NUM_CLIENTES) as executor:
    futures = [
        executor.submit(treinar_cliente_e_criptografar, i, clientes_X[i], state_dict_atual)
        for i in range(NUM_CLIENTES)
    ]
    for future in concurrent.futures.as_completed(futures):
        pesos_criptografados_clientes.append(future.result())

print("\n--- 4. Agregação Federada CEGA (Servidor Central) ---\n")
print("Servidor calculando operações sobre dados cifrados (FHE)...")
inicio_agregracao = time.time()

# Soma homomórfica
soma_cifrada = pesos_criptografados_clientes[0]
for i in range(1, NUM_CLIENTES):
    soma_cifrada += pesos_criptografados_clientes[i]

# Média homomórfica
media_cifrada = soma_cifrada * (1.0 / NUM_CLIENTES)

tempo_agreg = time.time() - inicio_agregracao
print(f"-> Agregação matemática FHE finalizada em {tempo_agreg:.2f}s.")

print("\n--- 5. Descriptografia e Inferência ---\n")
# Cliente decifra e restaura a topologia
pesos_planos_agregados = media_cifrada.decrypt()

# Restauração otimizada dos pesos para o modelo global
tensor_pesos = torch.tensor(pesos_planos_agregados, dtype=torch.float32).to(device)
vector_to_parameters(tensor_pesos, modelo_global.parameters())
print("-> Modelo Global atualizado com os pesos agregados!")
```